

**Universität Stuttgart**  
Master of Science

# Leader Elections

Marco Aiello

# Leader Elections

---

- Want to designate a unique processor as a leader, i.e., the coordinator of a task (breaking a deadlock, providing a tiebreaker, logging, main server, gateway, etc.).
- The network nodes communicate to make a decision based on a common criterion that breaks the symmetry among them.
- No birds-eye view of the system, often no knowledge of the size of the system, just channels to communicate
- We consider algorithms that are not crash fault-tolerant and others that are
- The topology is often organized as a ring.

# Algorithms' properties

---

- **Safety:** At most one  $p_i$  is a leader at any given moment:

$$\forall i, j \ i \neq j : \neg(\text{is\_leader}(i) \wedge \text{is\_leader}(j))$$

- **Liveness/Progress:** Eventually, all  $p_i$  are either the leader or not, all the non faulty ones have voted, and at least one non faulty  $p_i$  is the leader:

$$\forall i : \text{voted}(p_i) \wedge \exists j \text{is\_leader}(p_j)$$

# Basic definitions

---

- Nodes call *elections* (one, many, all), nodes *participate* in an election and must vote.
- Every process terminates in one of two final states: *elected* or *non-elected*
- In every admissible execution, one and only one process will be in the elected state and all others in the non-elected one
- A ring is *anonymous* if the processes do not have a unique identifier associated with them. In non-anonymous rings, the *vote\_id* must be different for all nodes and the id space is totally ordered (typically lexicographic order).
- An algorithm is *uniform* if the number of nodes in the ring is known to the processes, *non-uniform* otherwise

## Bad news...

---

- **Theorem** There is no anonymous leader election algorithm  $A$  for asynchronous ring systems. (even a more “constrained” version of the theorem is valid)
- **Theorem** There is no anonymous leader election algorithm  $A$  for nonuniform synchronous ring systems. (proof ab absurdum)
- **Lemma** For every round  $k$  of the admissible execution of  $A$  in the ring  $R$ , the states of all processes at the end of round  $k$  are the same. (proof by induction)

Therefore if one state machine is in the elected state, so are all the others. The second theorem implies the first one.

# Ring Algorithms

---

- Consider nodes organized in a ring
- We assume no crashes, non-uniform
- Asynchronous with FIFO channels

# LeLann-Chang-Roberts (LCR) algorithm

---

```
boolean participant = false;  
int leader_id = null;
```

## To initiate an election:

```
send(ELECTION(my_id));  
participant = true;
```

## Upon receiving a message ELECTION(*j*):

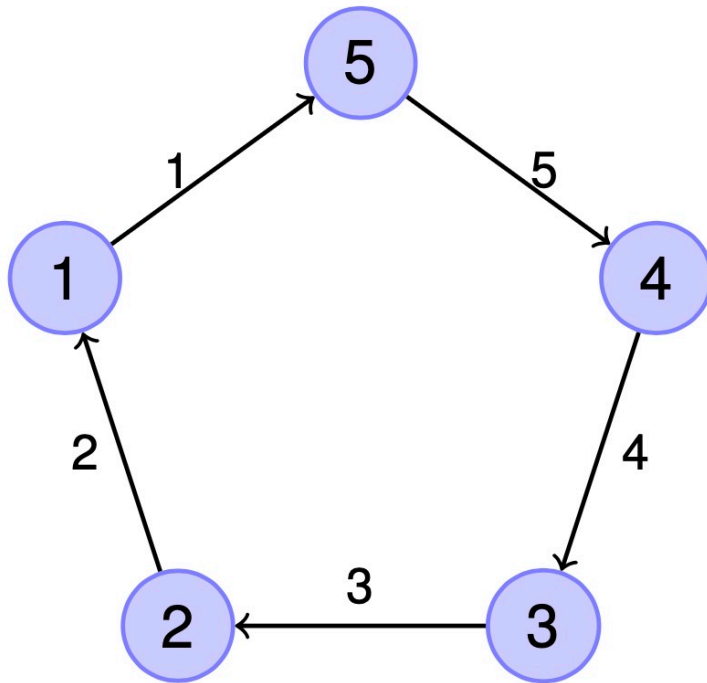
- if ( $j > my\_id$ ) then send(ELECTION(*j*));
- if ( $my\_id = j$ ) then send(LEADER(*my\_id*));
- if  $((my\_id > j) \vee (\neg participant))$  then  
    send(ELECTION(*my\_id*));  
    participant = true;

## Upon receiving a message LEADER(*j*):

```
leader_id = j;  
if ( $my\_id \neq j$ ) then send(LEADER(j));
```

# Example

---

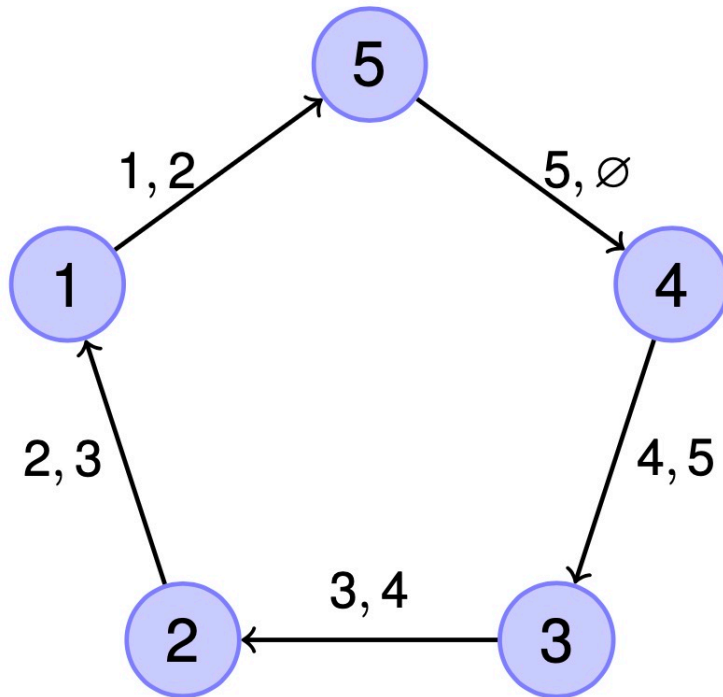


| Message ID | Times sent |
|------------|------------|
| <1>        | 1          |
| <2>        | 1          |
| <3>        | 1          |
| <4>        | 1          |
| <5>        | 1          |
| Total      | 5          |



## Example

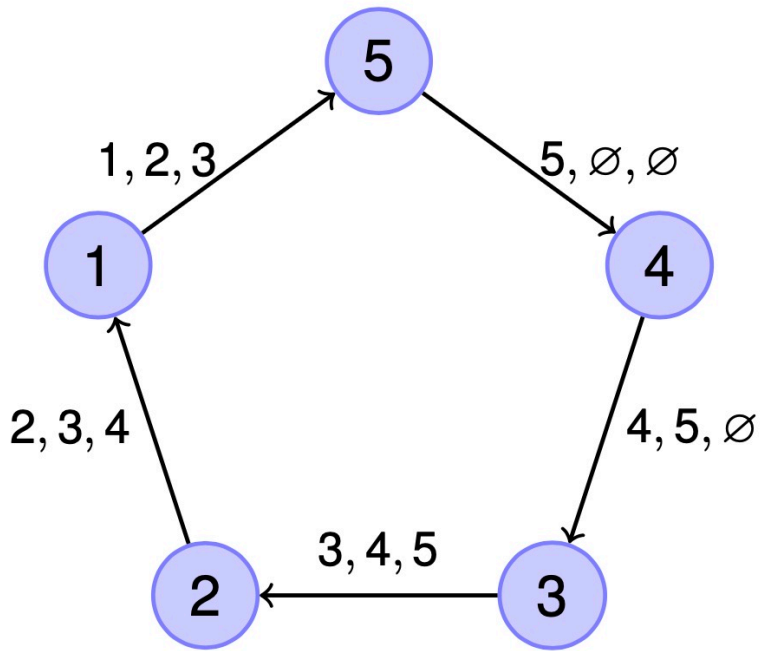
---



| Message ID | Times sent |
|------------|------------|
| <1>        | 1          |
| <2>        | 2          |
| <3>        | 2          |
| <4>        | 2          |
| <5>        | 2          |
| Total      | 9          |

## Example

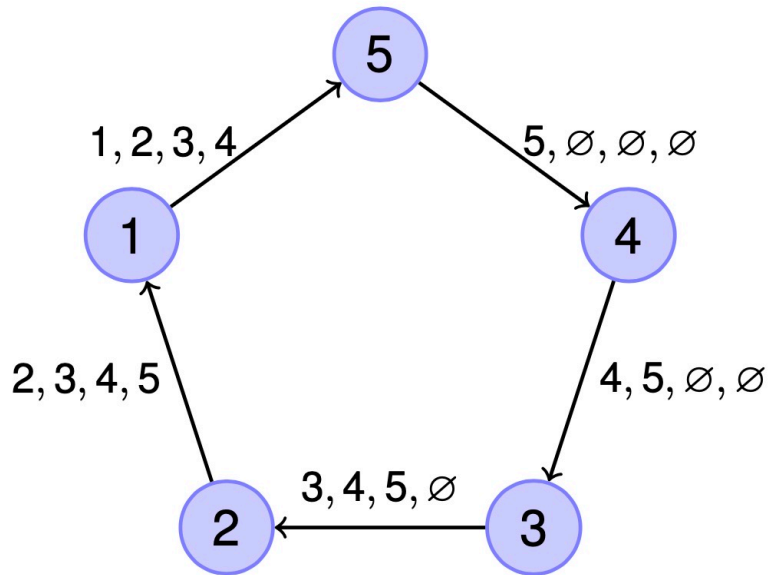
---



| Message ID | Times sent |
|------------|------------|
| <1>        | 1          |
| <2>        | 2          |
| <3>        | 3          |
| <4>        | 3          |
| <5>        | 3          |
| Total      | 12         |

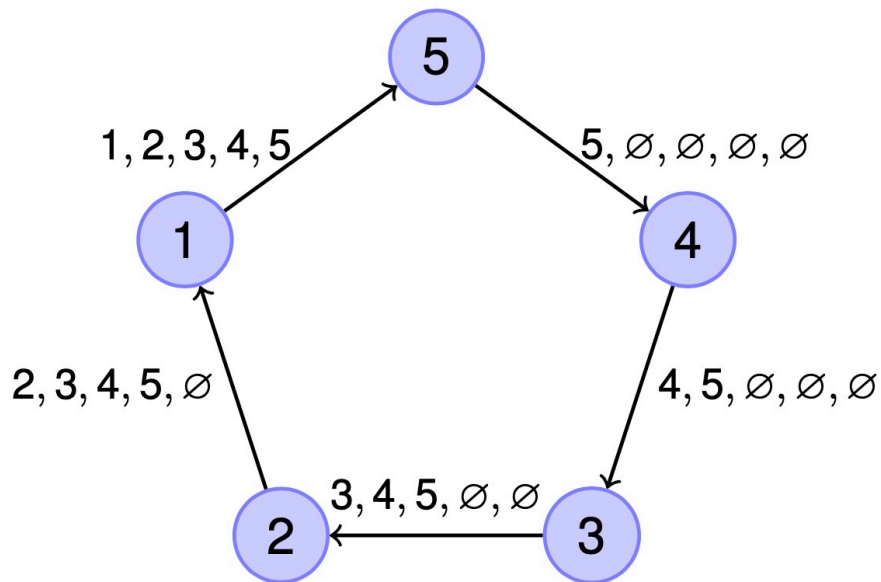
# Example

---



| Message ID | Times sent |
|------------|------------|
| <1>        | 1          |
| <2>        | 2          |
| <3>        | 3          |
| <4>        | 4          |
| <5>        | 4          |
| Total      | 14         |

## Example



| Message ID | Times sent |
|------------|------------|
| <1>        | 1          |
| <2>        | 2          |
| <3>        | 3          |
| <4>        | 4          |
| <5>        | 5          |
| Total      | 15         |

Leader announces itself: another 5 messages. Total: **20 messages**

# Message complexity: Worst case

---

- Complexity depends on node arrangement:
  - The largest id always travels all around the ring ( $n$  msgs).
  - 2<sup>nd</sup> largest id travels until reaching the largest.
  - 3<sup>rd</sup> largest id travels until reaching the largest or second largest.
  - ...
- Worst way to arrange the ids is in decreasing order (and all  $p_i$  s are initiators): 2<sup>nd</sup> largest causes  $n - 1$  messages, 3<sup>rd</sup> largest causes  $n - 2$ , and so on
- Number of messages is  $(n + (n - 1) + \dots + 1) + n = \frac{n(n + 1)}{2} + n$
- Worst case complexity is thus  $O(n^2)$

# Message complexity: Average

---

- Consider all possible ring permutations, there are  $n!$
- Each id makes 1 step  $n!$  times
- Each id makes a  $k^{th}$  step if it is the largest among all its neighbours from  $p_{i+1}$  to  $p_{i+k-1}$ :  $\Pr(\text{max\_among\_k}) = \frac{1}{k}$

- Add  $n!n$  times for the leader announcement

- Average number of message is

$$\begin{aligned} \frac{1}{n!} n \left( (n! + \frac{n!}{2} + \dots + \frac{n!}{n}) + n! \right) &= n \left( 1 + \frac{1}{2} + \dots + \frac{1}{n} \right) + n = \\ &= O(n)O(\log n) = O(n \log n) \end{aligned}$$

# Hirschberg-Sinclair Algorithm

---

- Aiming at an algorithm with worst case complexity  $O(n \log n)$  not just average
- Elections are done in neighbourhoods of increasing size in several phases; the size of the neighbourhood doubles in each phase
- When a node sees its own id traveling back to itself from the other side of the neighbourhood it can declare itself as the leader
- Initially all  $p_i$  initiate a candidacy (Phase 0) and consider themselves winners of the singleton neighbourhood.
- The (ELECTION( $id, k, d$ )) message contains the  $id$  of the candidate of the neighbourhood, the current phase  $k$ , and a hop counter  $d$

# Hirschberg-Sinclair Algorithm

---

After the election is initiated (Phase 0):

send(ELECTION( $my\_id, 0, 1$ )) to left and right;

Upon receiving a message ELECTION( $j, k, d$ ) from left [right]:

- if  $(j > my\_id) \wedge (d \leq 2^k)$  then send(ELECTION( $j, k, d + 1$ )) to the left [right];
- if  $(j > my\_id) \wedge (d = 2^k)$  then send(REPLY( $j, k$ )) to the right [left];
- if  $(my\_id = j)$  then announce itself as leader;

Upon receiving a message REPLY( $j, k$ ):

if  $(my\_i \neq j)$

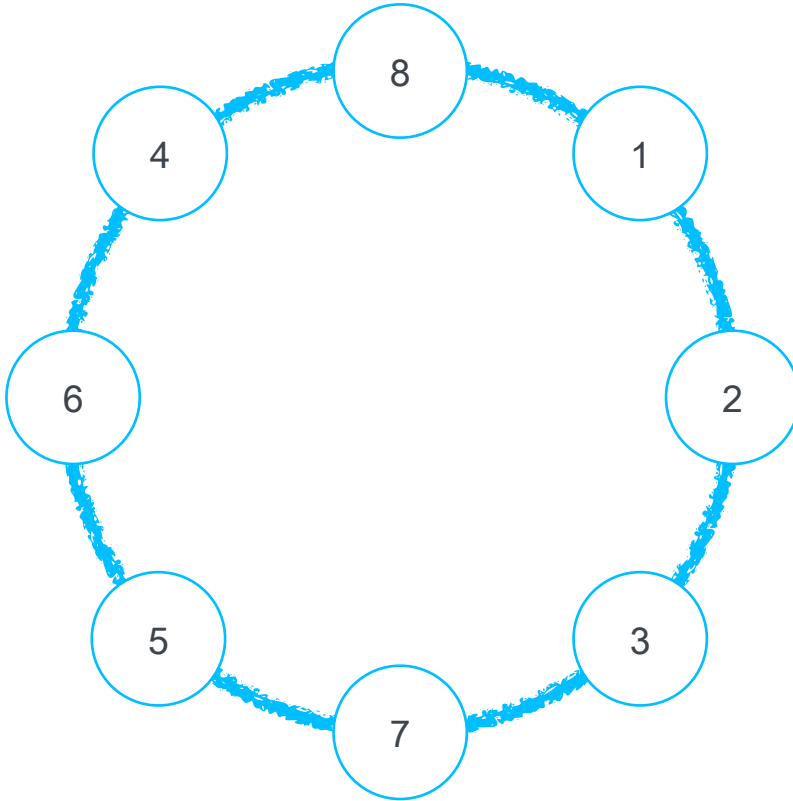
then send(REPLY( $j, K$ )) to the right [left];

else if (already received REPLY( $j, K$ )) then

send(ELECTION( $j, k + 1, 1$ )) to left and right.



# HS Example



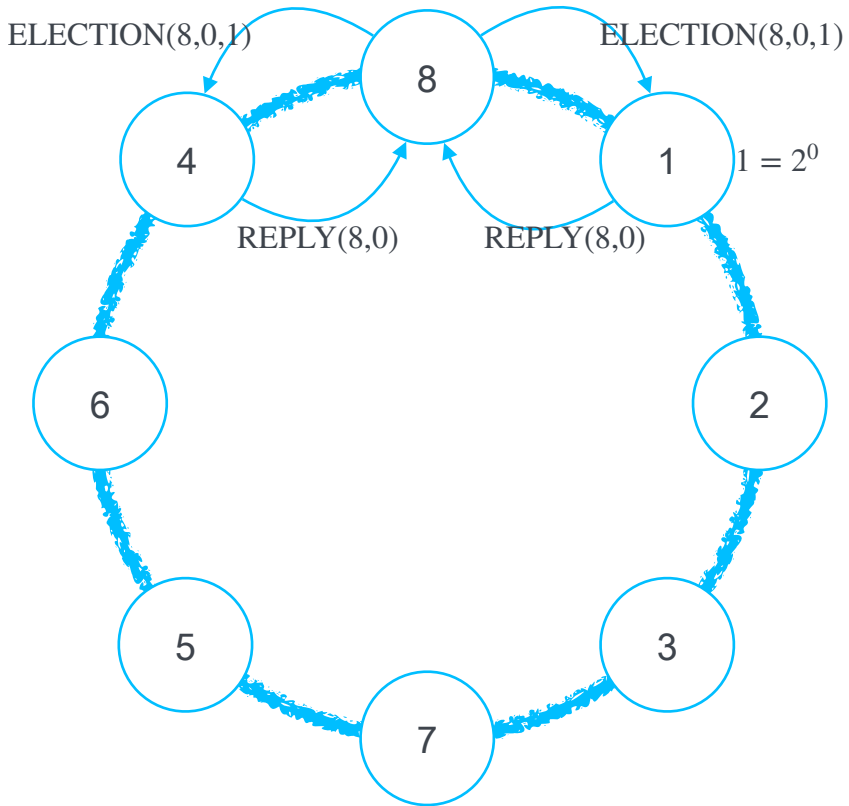
$$n = 8$$

number of phases is  $\lceil \log 8 \rceil + 1 = 4$

| Phase | Neighbourhood size | Leaders at end of round |
|-------|--------------------|-------------------------|
| 0     | 1                  | 6,7,8                   |
| 1     | 2                  | 7,8                     |
| 2     | 4                  | 8*                      |
| 3     | 8                  | 8                       |

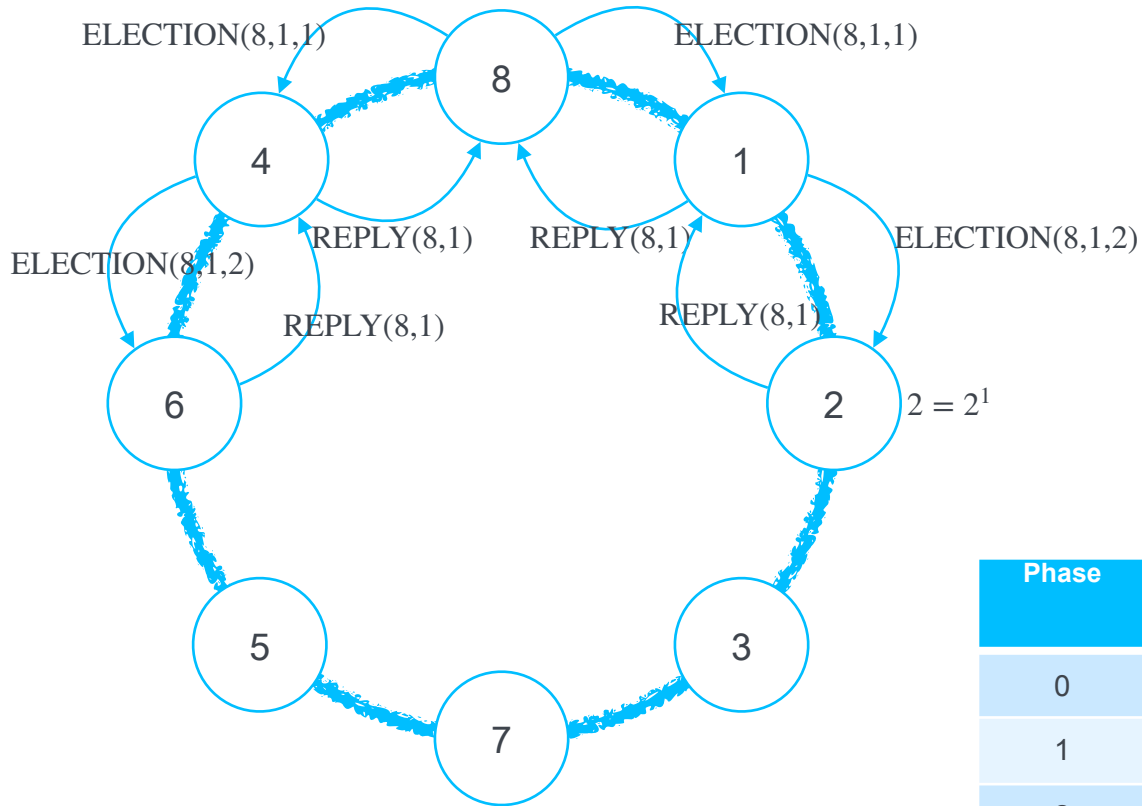
\* but it doesn't know yet

# HS Example: Phase 0 for node 8



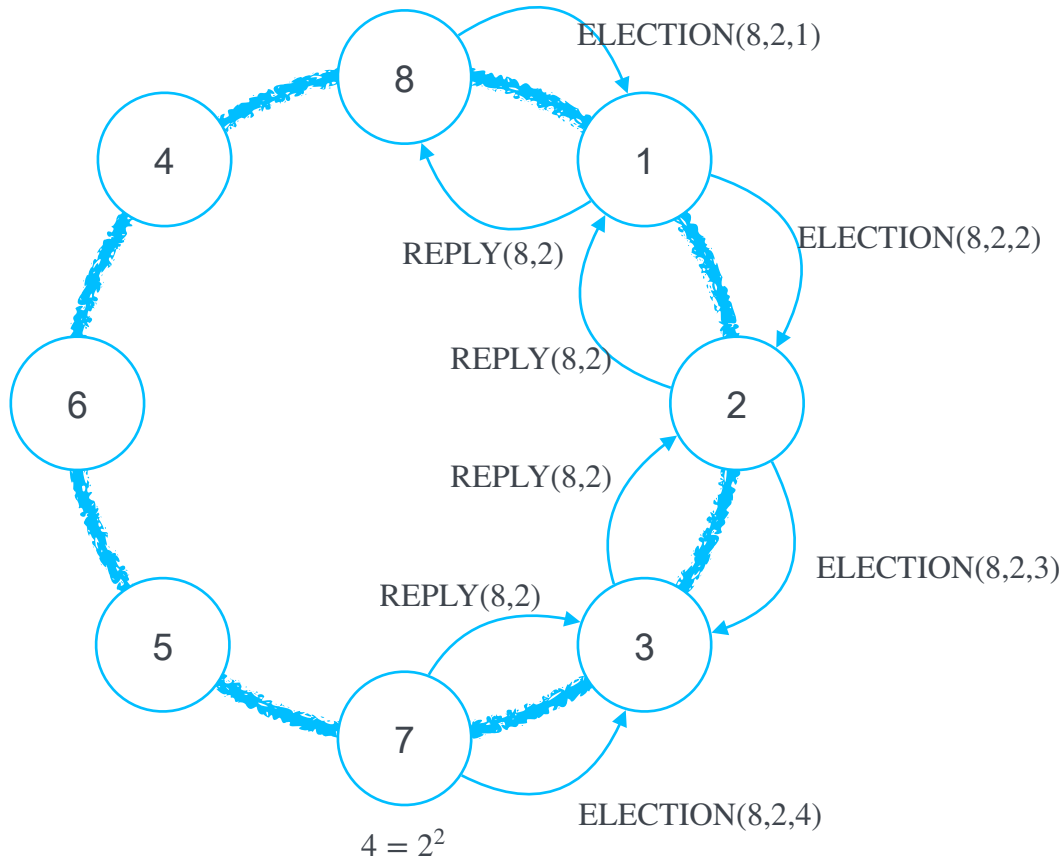
| Phase | Neighbourhood size | Leaders at end of round |
|-------|--------------------|-------------------------|
| 0     | 1                  | 6,7,8                   |
| 1     | 2                  | 7,8                     |
| 2     | 4                  | 8*                      |
| 3     | 8                  | 8                       |

# HS Example: Phase 1 for node 8

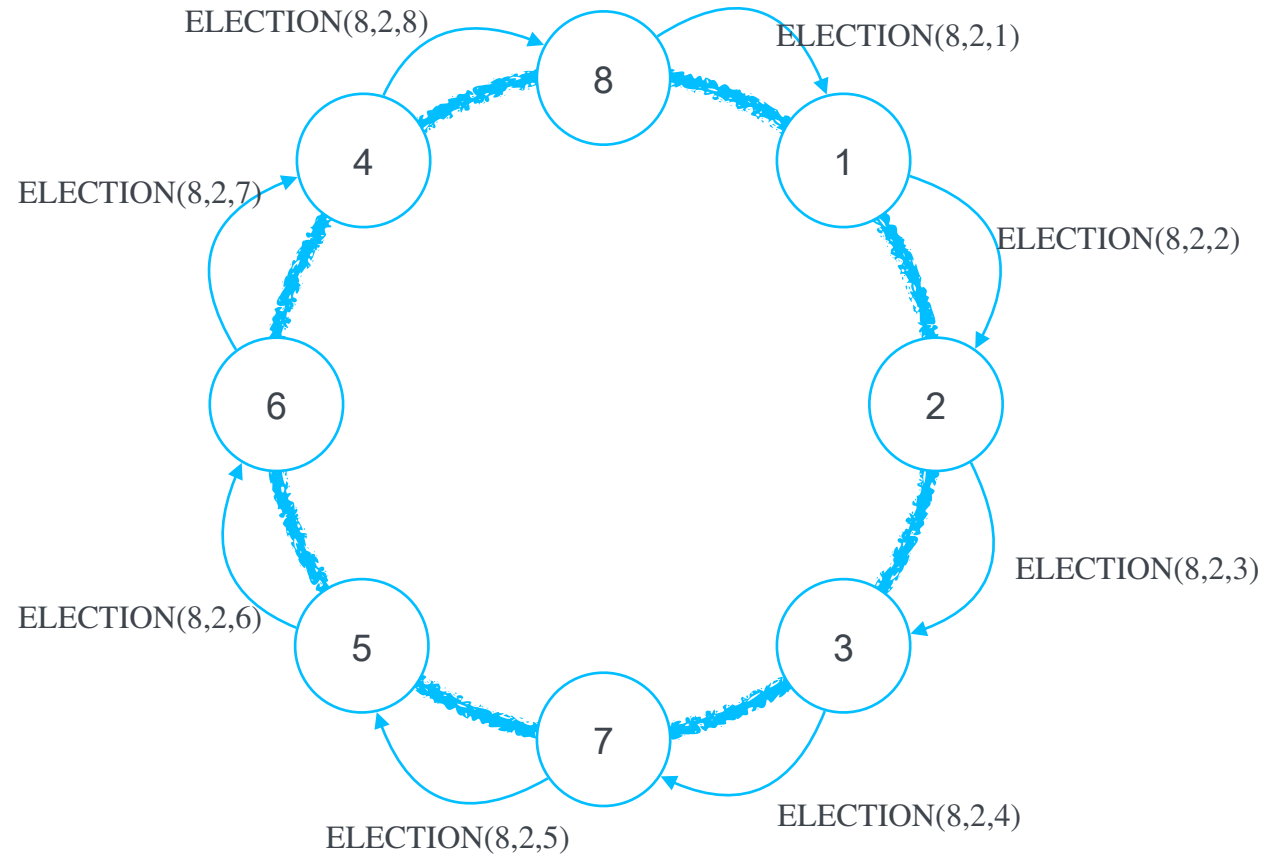


| Phase | Neighbourhood size | Leaders at end of round |
|-------|--------------------|-------------------------|
| 0     | 1                  | 6,7,8                   |
| 1     | 2                  | 7,8                     |
| 2     | 4                  | 8*                      |
| 3     | 8                  | 8                       |

## HS Example: Phase 2 for node 8 (right side only)



## HS Example: Phase 3 for node 8 (right side only with no replies)



# Message complexity

---

- At Phase  $k$  at most  $4 \cdot 2^k$  messages are circulated on behalf of a particular candidate (elections and replies).
- At Phase 0 there are  $n$  candidates
- At each Phase  $k$ , there are at most  $\left\lfloor \frac{n}{2^{k-1} + 1} \right\rfloor$  candidates
- In fact, the minimum distance between two winners at Phase  $k$  is  $2^k + 1$
- The total number of messages sent in any phase that is not the last one is
$$4 \left( 2^k \left\lfloor \frac{n}{2^{k-1} + 1} \right\rfloor + 1 \right) = 8n \left( \frac{2^{k-1}}{2^{k-1} + 1} \right) < 8n$$

# Message complexity

---

- The total number of phases till the leader is elected is  $\lceil \log n \rceil + 1$
- In the last phase  $2n$  messages are sent

$$\bullet 4n + \sum_{k=1}^{\lceil \log n \rceil - 1} \left( 4 \cdot 2^k \frac{n}{2^{k-1} + 1} \right) + 2n \leq 6n + 8n (\lceil \log n \rceil - 1)$$

- Worst case message complexity is therefore  $O(n \log n)$

# Time complexity

---

- The max time for each Phase  $k$  that is not the final is  $2 \cdot 2^k$
- The max total time required by phases 0 to  $k$  is
$$2(2^0 + 2^1 + \dots + 2^k) = 2(2^{k+1} - 1)$$
- The max total time required by all phases till the penultimate one is thus
$$2(2^{\lceil \log n \rceil + 1} - 1)$$
- Time for the final phase is  $n$
- Time complexity is therefore  $O(n)$
- **General Result:** Any leader election algorithm for asynchronous rings whose size is not known a priori has  $\Omega(n \log n)$  message complexity (holds also for unidirectional rings).



# The bully algorithm

---

- Assumptions: process can crash, message delivery reliable and synchronous, complete knowledge of `vote_id` of all other peers
- Synchronous: message turnaround time bound by  $T = T_{\text{transmission}} + T_{\text{process}}$
- Three types of messages:
  - *election* message: to call elections
  - *answer* message: to vote
  - *coordinator* message: to announce own acting as coordinator

# The Bully Algorithm (Garcia Molina, 1982)

---

- P broadcasts an election message (inquiry) to all other processes with higher process IDs.
- If P hears from no process with a higher process ID than itself, it wins the election and broadcasts victory.
- If P hears from a process with a higher ID, P waits a certain amount of time for that process to broadcast itself as the leader. If it does not receive this message in time, it re-broadcasts the election message.
- If P gets an election message (inquiry) from another process with a lower ID it sends an "I am alive" message back and starts new elections.

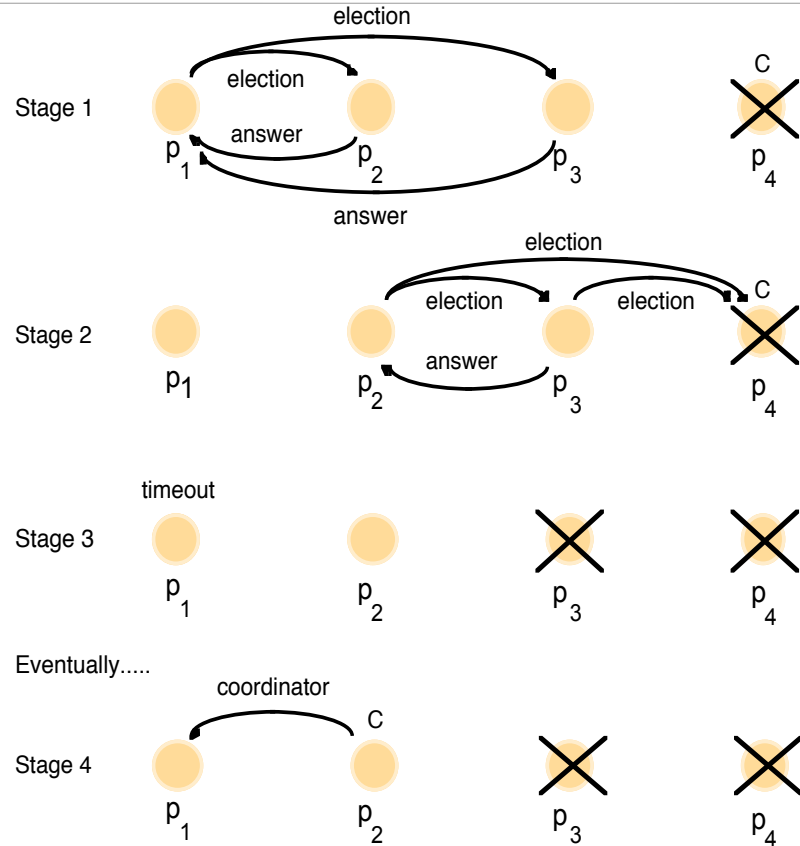
# The Bully Algorithm

---

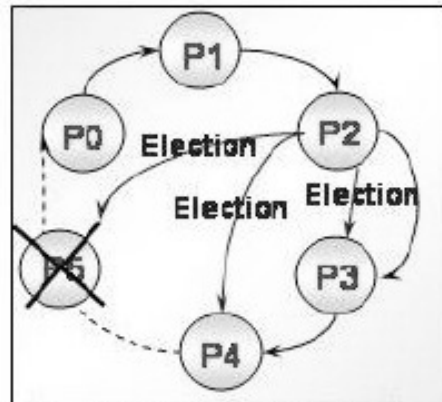
- Note that if P receives a victory message from a process with a lower ID number, it immediately initiates a new election. This is how the algorithm gets its name. A process with a higher ID number will bully a lower ID process out of the coordinator position as soon as it comes online.

# The bully algorithm

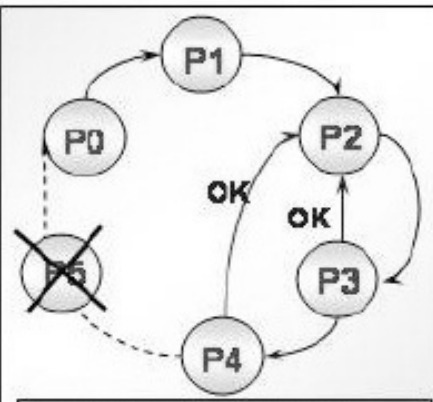
The election of coordinator  $p_2$ ,  
after the failure of  $p_4$  and then  $p_3$



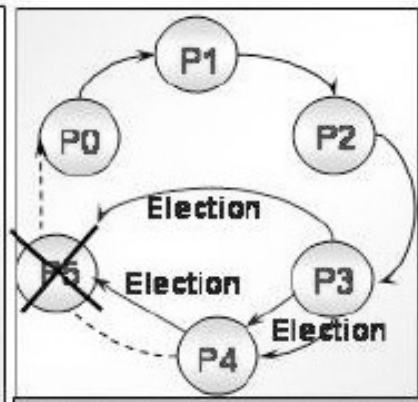
# The Bully Algorithm



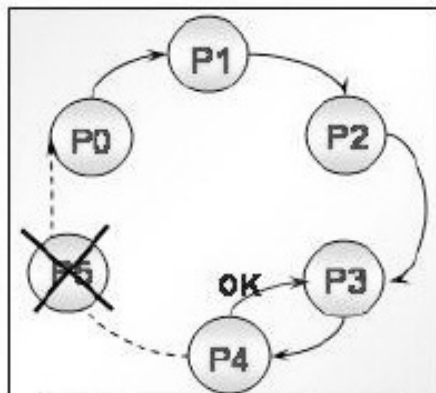
1. P2 initiates election



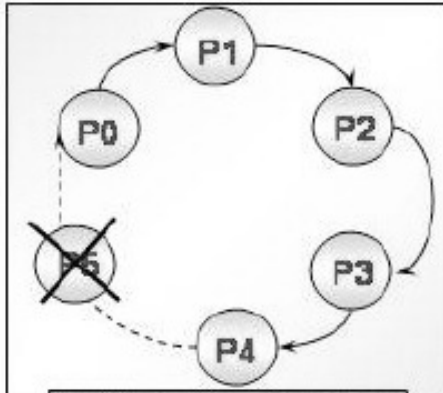
2. P2 receives "replies"



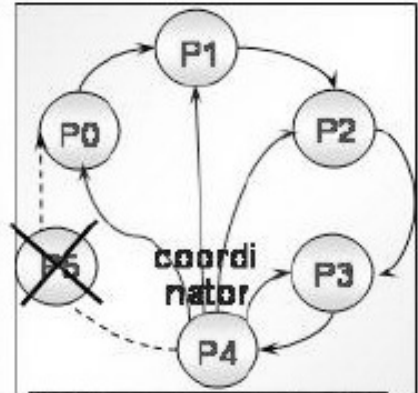
3. P3 & P4 initiate election



4. P3 receives reply



5. P4 receives no reply



5. P4 announces itself

# The bully algorithm

---

- The liveness condition is met by the reliable and bounded message transmission assumption
- Safety is met only if no new process starts during an election (because there is no guarantee on the order of message delivery if two processes send coordinator messages concurrently which may happen in case a new process starts)
- Message Complexity: each process sends to higher ID

$$\cdot \sum_{i=1}^{n-1} (n - i) = (n - 1) + (n - 2) + \dots + 1 = \frac{n(n - 1)}{2} = O(n^2) \text{ plus one}$$

round for the announcement, which doesn't change the class

- Time Complexity: worse case processes 1, 2, 3, ..., n-1 all sequentially time out and start elections  $O(n)$